

Reinforcement Learning: Policy Gradients (ESE 3990)

Instructor:
Antonio Loquercio

Fall 2026
University of Pennsylvania

Categorizing RL agents

Axis 1: How to make decisions

- Value Based
 - No Policy (Implicit)
 - Value/Q Function
- Policy Based
 - Policy
 - No Value/Q Function
- Actor Critic
 - Policy
 - Value Function

Today

Axis 2: Environment Representation

- Model-based
 - Policy and/or Value/Q Function
 - Model
- Model-free
 - Policy and/or Value/Q Function
 - No Model

Policy-Based Reinforcement Learning

- In the last lecture, we approximated the value of Q function using θ :

$$\begin{aligned}V_{\theta}(s) &\approx V^{\pi}(s) \\ Q_{\theta}(s, a) &\approx Q^{\pi}(s, a)\end{aligned}$$

- We then use these approximations to generate a policy
 - E.g., using ϵ -greedy

- In this lecture, we will directly parametrize **the policy**

$$\pi_{\theta}(s, a) = \mathbb{P}[a|s, \theta]$$

- We will focus on model-free reinforcement learning

Why Policy-Based RL?

- Advantages:
 - Better convergence properties
 - Effective in high-dimensional or continuous action spaces
 - Can learn stochastic policies
- Disadvantages:
 - Typically converges to a local rather than global optimal policy
 - Evaluating a policy is typically inefficient and high variance
 - Requires more samples (i.e., environment interactions) to converge

When to use Policy-Based RL?

- Problems for which learning the value function is challenging:
 - Continuous state-action spaces (as in robotics)
 - Very high-dimensional state and actions (as in LLMs)
- Problems where environment interactions are cheap
 - Agent interacts with a very fast simulator (millions of transitions/second)
- Problems where having a stochastic policy is important

Example: Rock-Paper-Scissor

- Two-player game of rock-paper-scissors

- Scissors beat paper
- Rock beats scissors
- Paper beats rock



- Consider policies for iterated rock-paper-scissors

- A deterministic policy is easily exploited
- A uniform random policy is optimal (i.e., Nash Equilibrium)

Example: Rock-Paper-Scissor

- Didn't we say that for any MDP, there is a unique deterministic policy that is optimal?
- Why, in this case, is the optimal policy not deterministic?
- In some problems, *algorithmic decisions can account for poor modeling assumptions.*



Policy Gradients: Core Ideas

- Assume we're in an MDP with $s \in S, a \in A, r \in R$.
- We parametrize the policy using the parameter θ : $\pi(a|s, \theta)$.
- By changing θ , we can control the agent's behavior.
- Each policy has a different value function, $V^\theta(s)$.
- **Goal.** Find the parameters θ that maximize the value function.

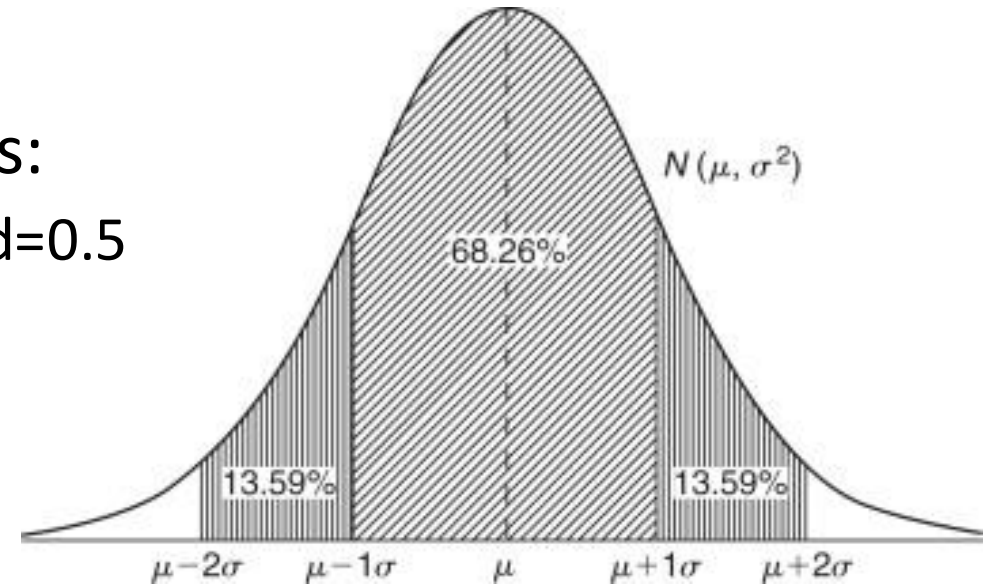
$$\theta^* = \arg \max_{\theta} V^\theta(s)$$

Example 1: Discrete States and Actions

- We have three possible states: $s \in \{0, \frac{1}{2}, 1\}$.
- There are *only two* possible actions, a_1 and a_2 .
- We can parametrize the policy as follows:
 - $\pi(a_1|s) = \theta s$
 - $\pi(a_2|s) = 1 - \theta s$
 - $\theta \in [0,1]$
- Why is this a bad parametrization?

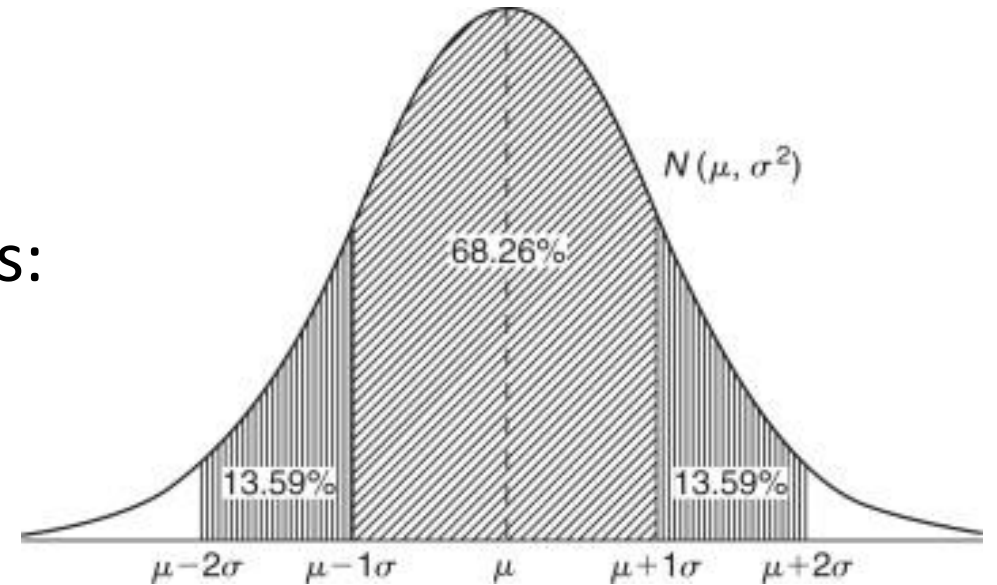
Example 2: Continuous States and Actions

- Continuous 1D state space: $s \in [0,1]$
- Continuous 1D action space: $a \in \mathbb{R}$
- We can parametrize the policy as follows:
 - Gaussian distribution with mean θs and std=0.5
 - $\pi(a|s) = \mathcal{N}(\theta s, 0.5)$



Example 3: Neural Networks

- Continuous 1D state space: $s \in [0,1]$
- Continuous 1D action space: $a \in \mathbb{R}$
- Neural network NN with parameters θ
- We can parametrize the policy as follows:
 - $\mu(s; \theta) = NN(s; \theta)$
 - $\pi(a|s) = \mathcal{N}(\mu(s; \theta), 0.5)$



Policy Objective Functions

- **Goal.** Find the parameters θ that maximize the value function.

$$\theta^* = \arg \max_{\theta} V^{\theta}(s)$$

- But for which state s ?
 - In episodic environments (w/ terminal state and/or $\gamma < 1$), we use the **expected value of the start states**

$$\theta^* = \arg \max_{\theta} \mathbb{E}_{s_0} [V^{\theta}(s_0)]$$

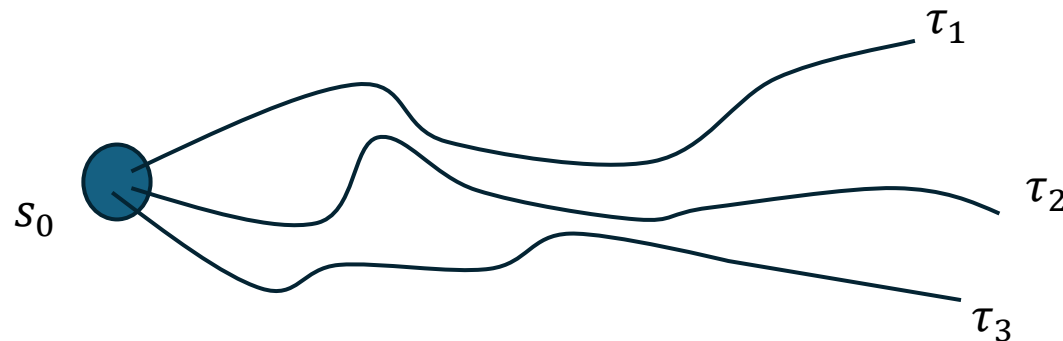
- In continuing environments (w/o terminal state and $\gamma = 1$), we use the **average value**

$$\theta^* = \arg \max_{\theta} \mathbb{E}_{d^{\pi}(s)} [V^{\theta}(s)]$$

- Where $d^{\pi}(s)$ is the stationary distribution of the (ergodic!) Markov chain

Policy Optimization

- Useful Identities (no γ for simplicity).
- $\theta^* = \arg \max_{\theta} J(\theta)$
- $= \arg \max_{\theta} \mathbb{E}_{s_0} [V^{\theta}(s_0)]$
- $= \arg \max_{\theta} \mathbb{E}_{(s_0, a_0, s_1, a_1, \dots, s_T) \sim \pi(\theta)} [\sum_{t=0}^{T-1} r(s_t, a_t)]$
- $= \arg \max_{\theta} \mathbb{E}_{\tau \sim p(\tau; \theta)} [R(\tau)]$



Policy Objective vs Value Objective

- In policy-based methods, we write a cost function and optimize it.

$$\theta^* = \arg \max_{\theta} \mathbb{E}_{\tau \sim p(\tau; \theta)} [R(\tau)]$$

- In Value-based methods (Sarsa, Q-learning, MC-Control), there is no *explicit* cost function to optimize.



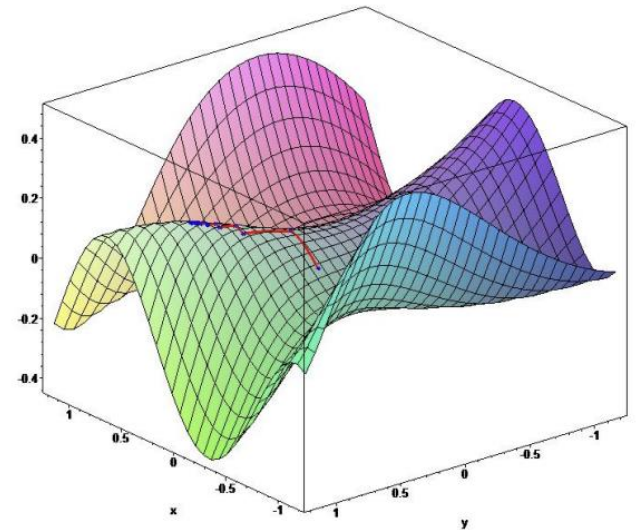
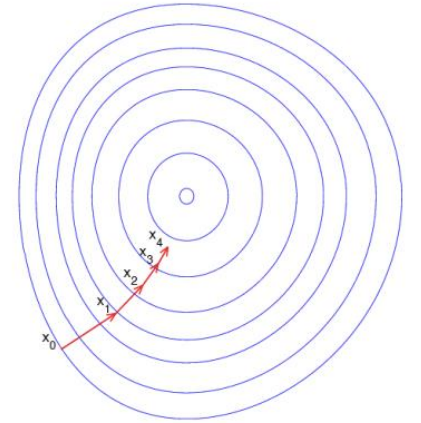
- We learn a value function with experience
- We act greedily with respect to the current value function
- Convergence to optimal policy is guaranteed (under some strict assumptions)

Policy Optimization: How to find θ^* ?

- **Gradient-free** algorithms
 - Brute force (try all possible θ , pick the best)
 - Genetic algorithms
 - ...
 - Do not exploit the sequential structure of the problem. Last resort.
- **Gradient-based** algorithms
 - Gradient-ascent
 - Quasi-Newton
 - ...
 - They exploit the problem's sequential structure and decomposition into sub-problems.
- We will focus on gradient ascent methods.

Policy Gradient

- Let $J(\theta)$ be the objective function
- Policy-gradient algorithms search for a local maximum of $J(\theta)$ by ascending the gradient of the policy w.r.t. parameters θ
$$\Delta\theta = \alpha \nabla_{\theta} J(\theta)$$
 - Where α is the scalar step size.
- $\nabla_{\theta} J(\theta)$ is called the **policy-gradient**



Why is Computing the Policy Gradient Hard?

- Consider, as an example, a differentiable function $J(\theta) = \theta^2$
 - What is the gradient of $J(\theta)$?
- Consider a one-step MDP: Biased-Coin Toss.
 - Always start at the same $s_0 = h$
- $J(\theta) = \mathbb{E}_{(s_0, a_0 = \pi_\theta(s_0), s_1)} [r(s_1)]$
- $= P[s_1 = h \mid a_0 = \pi_\theta(s_0)]r(h) + P[s_1 = t \mid a_0 = \pi_\theta(s_0)]r(t)$
- What is $\frac{\partial P[s_1 \mid \pi_\theta(s_0)]}{\partial \theta}$?



Why is Computing the Policy Gradient Hard?

- It requires differentiation through the environment!
- How can we differentiate through something we don't know?
 - **Approach 1**: Fit a differentiable model to the environment, and use that gradient instead (differentiable simulators).
 - **Approach 2**: Use an approximation of the gradient.

Score Function Identity

- $J(\theta) = \mathbb{E}_{x \sim P(x; \theta)}[f(x)]$
- $\nabla J(\theta) = \nabla \mathbb{E}_{x \sim P(x; \theta)}[f(x)]$
- $= \nabla \sum_{x \in X} p(x; \theta) f(x)$ (definition of expectation)
- $= \sum_{x \in X} \nabla p(x; \theta) f(x)$ (linearity of expectation)
- $= \sum_{x \in X} \frac{p(x; \theta)}{p(x; \theta)} \nabla p(x; \theta) f(x)$ (multiply by $\frac{p(x; \theta)}{p(x; \theta)} = 1$)
- $= \sum_{x \in X} p(x; \theta) \nabla \log p(x; \theta) f(x)$ (definition of log gradient)
- $= \mathbb{E}_{x \sim p(x; \theta)}[\nabla \log p(x; \theta) f(x)]$ (definition of expectation)

Score Function Identity

$$\nabla_{\theta} J(\theta) = \nabla_{\theta} \mathbb{E}_{x \sim P(x; \theta)} [f(x)] = \mathbb{E}_{x \sim p(x; \theta)} [\nabla_{\theta} \log p(x; \theta) f(x)]$$

- Transforms the *gradient of an expectation* to the *expectation of a gradient*.

- $\nabla_{\theta} J(\theta) = \nabla_{\theta} \mathbb{E}_{(s_0, a_0 = \pi_{\theta}(s_0), s_1)} [r(s_1)]$

- $= \nabla_{\theta} P[s_1 = h \mid a_0 = \pi_{\theta}(s_0)] r(h) + \nabla_{\theta} P[s_1 = t \mid a_0 = \pi_{\theta}(s_0)] r(t)$



- $\nabla_{\theta} J(\theta) = \mathbb{E}_{(s_0, a_0 = \pi_{\theta}(s_0), s_1)} [\nabla_{\theta} \log \pi(a_0 | s_0, \theta) r(s_1)] =$

- $= P[s_1 = h \mid a_0 = \pi_{\theta}(s_0)] \nabla_{\theta} \log \pi(a_0 | s_0, \theta) r(h) +$ (prob. heads)

- $+ P[s_1 = t \mid a_0 = \pi_{\theta}(s_0)] \nabla_{\theta} \log \pi(a_0 | s_0, \theta) r(t)$ (prob. tails)

- Only requires gradient of the policy (which we know), **not** of the env.

Score Function Identity

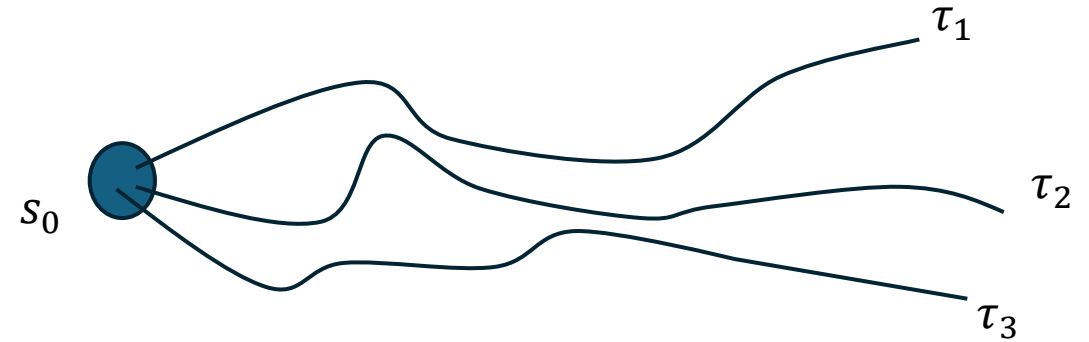
$$\nabla_{\theta} J(\theta) = \nabla_{\theta} \mathbb{E}_{x \sim P(x; \theta)} [f(x)] = \mathbb{E}_{x \sim p(x; \theta)} [\nabla_{\theta} \log p(x; \theta) f(x)]$$

- Transforms the gradient of an expectation to the expectation of a gradient.
- Why is this useful for us?
 - $J(\theta) = \mathbb{E}_{\tau \sim p(\tau; \theta)} [R(\tau)]$
 - $\nabla_{\theta} J(\theta) = \nabla_{\theta} \mathbb{E}_{\tau \sim p(\tau; \theta)} [R(\tau)] = \mathbb{E}_{\tau \sim p(\tau; \theta)} [\nabla_{\theta} \log(p(\tau; \theta)) R(\tau)]$
- It turns out that, exploiting the sequential nature of the problem, we can analytically compute $\nabla_{\theta} \log(p(\tau; \theta))$ without:
 - Differentiating over the environment
 - Even *knowing* the environment transition probabilities 

Simplifying the gradient of trajectories

- What does $p(\tau; \theta)$ represent?

- What is the probability of a trajectory τ given policy parameters θ



- What does $\nabla_{\theta} p(\tau; \theta)$ represent?

- The direction ϵ we change the parameters θ to maximize the probability of $p(\tau; \theta + \alpha\epsilon)$, for $\alpha \rightarrow 0$.
- Since $\log(p(\tau; \theta))$ is a strictly monotone function of $p(\tau; \theta)$, $\nabla_{\theta} \log(p(\tau; \theta))$ has the same meaning.

Simplifying the gradient of trajectories

- $\nabla_{\theta} \log(p(\tau; \theta)) =$
- $\nabla_{\theta} \log(p(s_0)\pi(a_0|s_0, \theta)p(s_1|s_0, a_0) \cdots p(s_T|s_{T-1}, a_{T-1}))$ (def. of $p(\tau; \theta)$)
- $= \nabla_{\theta} \log\left(p(s_0)\prod_{t=0}^{T-1}\pi(a_t|s_t, \theta)p(s_{t+1}|s_t, a_t)\right)$
- $= \nabla_{\theta} \left(\log(p(s_0)) + \sum_{t=0}^{T-1} \log \pi(a_t|s_t, \theta) + \log p(s_{t+1}|s_t, a_t)\right)$ (log of product = sum)
- $= \nabla_{\theta} \log(p(s_0)) + \sum_{t=0}^{T-1} \nabla_{\theta} (\log \pi(a_t|s_t, \theta)) + \nabla_{\theta} (\log p(s_{t+1}|s_t, a_t))$ (θ independent)
- $= \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi(a_t|s_t, \theta)$ (sum only gradients of actions!)

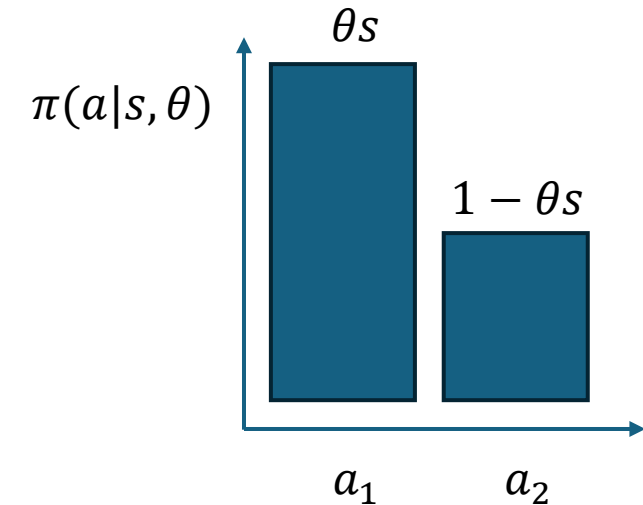
Simplifying the gradient of trajectories

$$\nabla_{\theta} \log(p(\tau; \theta)) = \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi(a_t | s_t, \theta)$$

- The environment transition probabilities disappear!
- Therefore, we don't need to differentiate with respect to the environment (or even know the transition probabilities) to compute the gradient of trajectories.
 - Isn't that crazy? 😊

Simplifying the gradient of trajectories

- What does $\pi(a_t|s_t, \theta)$ represent?
 - The probability of the policy selecting action a_t when being in state s_t using parameters θ .
- What does $\nabla_{\theta} \pi(a_t|s_t, \theta)$ represent?
 - The direction ϵ we change the parameters θ to maximize the probability $\pi(a_t|s_t, \theta + \alpha\epsilon)$, for $\alpha \rightarrow 0$
 - Same meaning for $\nabla_{\theta} \log \pi(a_t|s_t, \theta)$
 - **Easy to compute**: $\pi(a_t|s_t, \theta)$ is generally a differentiable function of θ .



Putting all together

$$\begin{aligned}\nabla_{\theta} J(\theta) &= \nabla_{\theta} \mathbb{E}_{\tau \sim p(\tau; \theta)} [R(\tau)] \\ &= \mathbb{E}_{\tau \sim p(\tau; \theta)} [\nabla_{\theta} \log(p(\tau; \theta)) R(\tau)] \\ &= \mathbb{E}_{\tau \sim p(\tau; \theta)} \left[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi(a_t | s_t, \theta) R(\tau) \right]\end{aligned}$$

- **Why is this convenient?**
- We can compute an approximation of the gradient through samples

$$\hat{\nabla}_{\theta} J(\theta) = \frac{1}{K} \sum_{k=0}^{K-1} \left[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi(a_t^k | s_t^k, \theta) R^k(\tau) \right]$$

Algorithm Summary

- Initialize θ arbitrarily
- Until tired/converged:
 - Run the policy $\pi(a|s, \theta)$ for K episodes of length T
 - Compute policy gradient $\hat{\nabla}_{\theta} J(\theta) = \frac{1}{K} \sum_{k=0}^{K-1} \left[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi(a_t^k | s_t^k, \theta) R^k(\tau) \right]$
 - $\theta \leftarrow \theta + \alpha \hat{\nabla}_{\theta} J(\theta)$
- Return θ

Simplifying the Policy Gradient Further

$$\nabla_{\theta} J(\theta) = \nabla_{\theta} \mathbb{E}_{\tau \sim p(\tau; \theta)} [R(\tau)] = \mathbb{E}_{\tau \sim p(\tau; \theta)} \left[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi(a_t | s_t, \theta) R(\tau) \right]$$

- Let's compute the gradient w.r.t a **single reward term**, at $t = t'$

$$\nabla_{\theta} \mathbb{E}_{\tau \sim p(\tau; \theta)} [r(s_{t'}, a_{t'})] = \mathbb{E}_{\tau \sim p(\tau; \theta)} \left[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi(a_t | s_t, \theta) r(s_{t'}, a_{t'}) \right]$$

- $= \mathbb{E}_{\tau \sim p(\tau; \theta)} \left[\sum_{t=0}^{t'} \nabla_{\theta} \log \pi(a_t | s_t, \theta) r(s_{t'}, a_{t'}) \right]$ (stop the sum at t')

Simplifying the Policy Gradient Further

$$\nabla_{\theta} \mathbb{E}_{\tau \sim p(\tau; \theta)} [r(s_{t'}, a_{t'})] = \mathbb{E}_{\tau \sim p(\tau; \theta)} \left[\sum_{t=0}^{t'} \nabla_{\theta} \log \pi(a_t | s_t, \theta) r(s_{t'}, a_{t'}) \right]$$

- Getting back the original cost by summing all $r(s_{t'}, a_{t'})$
- $\sum_{t'=0}^{T-1} \nabla_{\theta} \mathbb{E}_{\tau \sim p(\tau; \theta)} [r(s_{t'}, a_{t'})]$
- $\nabla_{\theta} \sum_{t'=0}^{T-1} \mathbb{E}_{\tau \sim p(\tau; \theta)} [r(s_{t'}, a_{t'})]$ (linearity of gradient)
- $= \nabla_{\theta} \mathbb{E}_{\tau \sim p(\tau; \theta)} [\sum_{t'=0}^{T-1} r(s_{t'}, a_{t'})]$ (linearity of expectation)
- $= \nabla_{\theta} \mathbb{E}_{\tau \sim p(\tau; \theta)} [R(\tau)]$ (original objective)

Simplifying the Gradient Further (Brace for Impact)

- $\nabla_{\theta} \mathbb{E}_{\tau \sim p(\tau; \theta)} [R(\tau)] = \sum_{t'=0}^{T-1} \nabla_{\theta} \mathbb{E}_{\tau \sim p(\tau; \theta)} [r(s_{t'}, a_{t'})]$
- $= \sum_{t'=0}^{T-1} \mathbb{E}_{\tau \sim p(\tau; \theta)} \left[\sum_{t=0}^{t'} \nabla_{\theta} \log \pi(a_t | s_t, \theta) r(s_{t'}, a_{t'}) \right]$ (slide 28)
- $= \mathbb{E}_{\tau \sim p(\tau; \theta)} \left[\sum_{t'=0}^{T-1} \sum_{t=0}^{t'} \nabla_{\theta} \log \pi(a_t | s_t, \theta) r(s_{t'}, a_{t'}) \right]$ (slide 29)
- $= \mathbb{E}_{\tau \sim p(\tau; \theta)} \left[\sum_{t=0}^{T-1} \left(\nabla_{\theta} \log \pi(a_t | s_t, \theta) \sum_{t'=t}^{T-1} r(s_{t'}, a_{t'}) \right) \right]$ (rearrange terms)
- $= \mathbb{E}_{\tau \sim p(\tau; \theta)} \left[\sum_{t=0}^{T-1} \left(\nabla_{\theta} \log \pi(a_t | s_t, \theta) \mathbf{G}_t \right) \right]$ (return definition)

Summary: Policy Gradient Theorem

- We have shown that for episodic MDPs:

$$\nabla_{\theta} J(\theta) = \nabla_{\theta} \mathbb{E}_{\tau \sim p(\tau; \theta)} [R(\tau)] = \mathbb{E}_{\tau \sim p(\tau; \theta)} \left[\sum_{t=0}^{T-1} (\nabla_{\theta} \log \pi(a_t | s_t, \theta) \textcolor{red}{G}_t) \right]$$

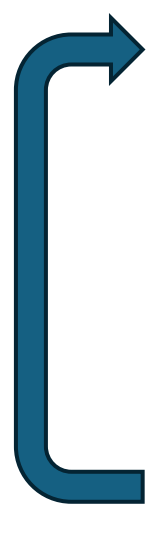
- Furthermore, for any differentiable $\pi_{\theta}(s, a)$, in an ergodic continuing MDP, we can show that:

$$\begin{aligned} \nabla_{\theta} J(\theta) &= \nabla_{\theta} \mathbb{E}_{d^{\pi}(s)} [V^{\theta}(s)] \\ &= \mathbb{E}_{\textcolor{red}{s} \sim \textcolor{red}{d}^{\pi}(\textcolor{red}{s}; \theta), \textcolor{red}{a} \sim \pi(\textcolor{red}{a} | \textcolor{red}{s}, \theta)} [\nabla_{\theta} \log \pi(a | s, \theta) \textcolor{red}{Q}^{\theta}(\textcolor{red}{s}, \textcolor{red}{a})] \end{aligned}$$

A (Better) Algorithm

Previous

- Initialize θ arbitrarily
- Until tired:

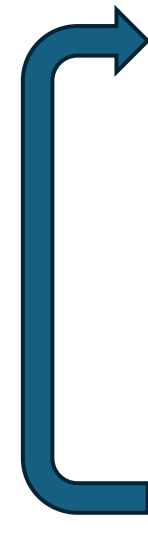


- Run the policy $\pi(a|s, \theta)$ for K episodes of length T
- Compute policy gradient
$$\hat{\nabla}_{\theta} J(\theta) = \frac{1}{K} \sum_{k=0}^{K-1} \left[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi(a_t^k | s_t^k, \theta) R^k(\tau) \right]$$
- $\theta \leftarrow \theta + \alpha \hat{\nabla}_{\theta} J(\theta)$

- Return θ

New (REINFORCE, '92)

- Initialize θ arbitrarily
- Until tired:



- Run the policy $\pi(a|s, \theta)$ for K episodes of length T
- Compute policy gradient
$$\hat{\nabla}_{\theta} J(\theta) = \frac{1}{K} \sum_{k=0}^{K-1} \left[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi(a_t^k | s_t^k, \theta) G_t^k \right]$$
- $\theta \leftarrow \theta + \alpha \hat{\nabla}_{\theta} J(\theta)$

- Return θ

Intuitive Interpretation

$$\nabla_{\theta} \log \pi(a_t^k | s_t^k, \theta) \quad G_t^k$$

- **Intuition:**

- Increase the probability of actions according to the return *that follows them*.
- *The better the return*, the more the policy gradient aims to improve it.
- Maximizing the probability of action sequences that lead to good returns and minimizing the one that leads to poor returns.
- Policy-based methods formalize trial-and-error!

Geometric Interpretation

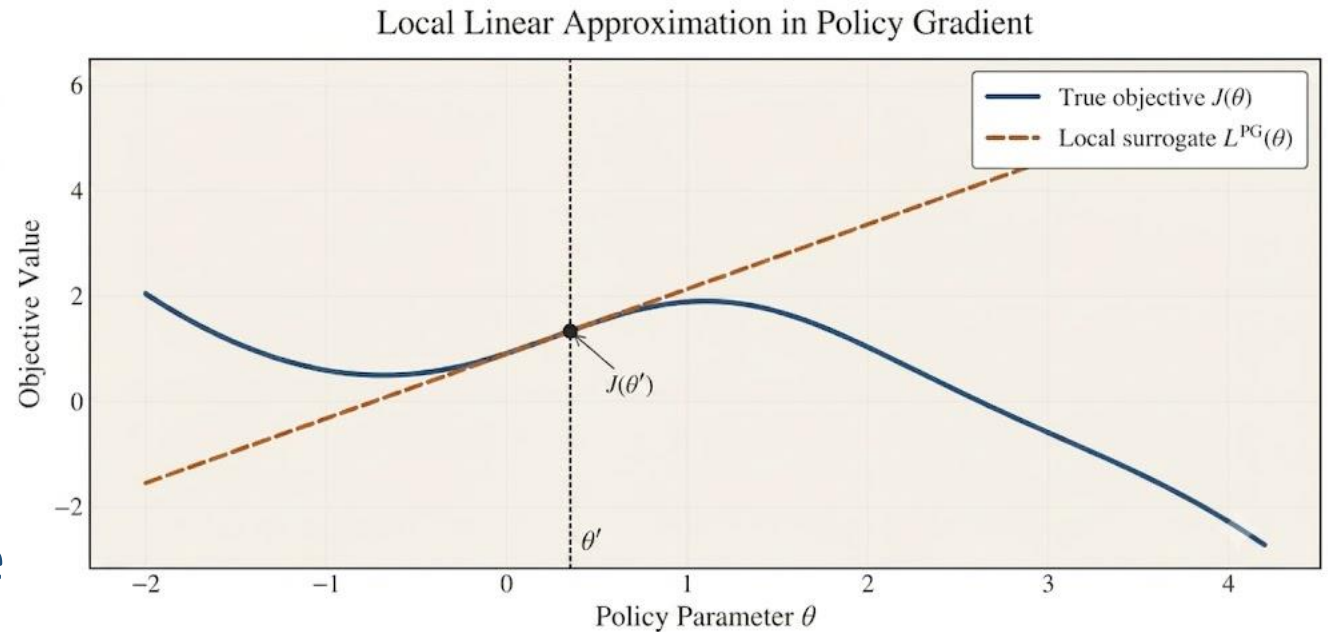
- True Objective

$$J(\theta) = \mathbb{E}_{\tau \sim p(\tau; \theta)} [R(\tau)]$$

- Local Surrogate Objective

$$L(\theta) = \sum_{t=0}^{T-1} (\log \pi(a_t | s_t, \theta) G_t)$$

- The **local surrogate** and **true objective** have the same gradient at the policy parameters θ' that collected the trajectories.



Reinforce: Advantages and Limitations

Advantages

- Extremely easy to use
- Good performance in complex problems (after enough tuning)

Limitations

- High-variance
- Very hard to tune

Next Class: Methods to address these limitations

